800行代码实现 Open Claw 的 Tool、消息总线、子Agent管理架构

原创 会员技术团队 大淘宝技术 2026年4月24日 01:29 浙江 7人



这篇文章记录对 Open Claw 中 Tool、消息总线和子 Agent 管理架构的研究学习，以及一个最小可运行实现。

本文想说明的技术观点是对于 Tool 调用、消息分发、子 Agent 管理这三类 Agent 系统里的核心组件，优先采用薄抽象、显式控制流和贴近模型 API 的实现方式，往往比引入多层中间件更容易获得工程上的确定性。系统边界更清晰，运行路径更容易追踪，问题更容易定位，也更适合作为后续扩展 Memory、调度和持久化能力的基础。

01 引言

这是一个基于 Anthropic Claude API 的 Agent 框架，用 TypeScript 编写，运行在单进程 Node.js 环境中。

本文记录其中四个核心模块的实现：工具系统（Tool layer）、消息总线（MessageBus）、子 Agent 管理（SubagentManager）、REPL 主循环。不涉及上层 Bot 接入层、持久化、Context / Memory 系统。

框架不依赖 LangChain 或其他 Agent 框架，直接基于 Anthropic SDK 构建。选择这条路的原因很简单：中间层越薄，调试越容易，对 API 行为的控制越精确。

02 基础设施：Tool 抽象与 ToolRegistry

Tool 抽象类

一个工具由四个要素组成：`name`、`description`、`input_schema`、`execute`。

```
export abstract class Tool {
  abstract readonly name: string;
  abstract readonly description: string;
```

```
  abstract readonly input_schema: AnthropicTool["input_schema"];
  abstract execute(args: Record<string, unknown>): Promise<unknown>;
  toSchema(): AnthropicTool {
    return {
      name: this.name,
      description: this.description,
      input_schema: this.input_schema,
    };
  }
}
```

`input_schema` 的类型直接取自 `@anthropic-ai/sdk` 的 `Tool` 类型定义。`toSchema()` 将实例转换为 Anthropic API 要求的 function calling schema。没有中间层转换，SDK 类型就是唯一的 schema 定义。

这里有一个刻意的取舍：schema 使用运行时普通对象定义，而非 Zod 等库。好处是零额外依赖、直接对齐 SDK 类型。代价是没有运行时参数校验——LLM 传入的参数如果类型不对，只能靠 `execute` 内部的 `as` 断言和实际调用时的错误来兜底。对于当前规模，这个取舍可以接受。

ToolRegistry

注册表本身是一个 `Map<string, Tool>`：

```
export class ToolRegistry {
  private tools = new Map<string, Tool>();
  register(tool: Tool) {
    this.tools.set(tool.name, tool);
  }
  async execute(name: string, args: Record<string, unknown>) {
    const tool = this.tools.get(name);
    if (!tool) throw new Error(`Tool "${name}" not found`);
    return tool.execute(args);
  }
  getToolDefinition(): AnthropicTool[] {
    return Array.from(this.tools.values()).map((tool) => tool.toSchema
  }
  exclude(names: string[]): ToolRegistry {
    const excludeSet = new Set(names);
    const filtered = new ToolRegistry();
    for (const [name, tool] of this.tools) {
      if (!excludeSet.has(name)) {
        filtered.register(tool);
      }
```

```
    }
    return filtered;
  }
}
```

`exclude()` 是为子 Agent 设计的。子 Agent 不应该持有 `spawn`（避免递归创建子 Agent）、`message`（避免直接向用户发消息）等工具，所以需要从主 Agent 的工具集中排除特定工具，生成一个受限子集。`exclude()` 返回新的 `ToolRegistry` 实例，不修改原注册表。



文件操作

ReadFileTool — 读取文件内容，动态 `import("node:fs/promises")` 加载模块。
WriteFileTool — 写入文件。写入前调用 `mkdir(dirname(path), { recursive: true })` 自动创建父目录，避免因目录不存在而失败。
EditFileTool — 精确文本替换。核心逻辑：

```
const occurrences = content.split(oldText).length - 1;
if (occurrences === 0) {
  return `Error: old_text not found in ${filePath}`;
}
if (occurrences > 1) {
  return `Warning: old_text found ${occurrences} times in ${filePath}.
}
const updated = content.replace(oldText, newText);
```

强制唯一匹配：出现 0 次报错，超过 1 次拒绝写入并要求提供更精确的文本片段。这个设计是为了防止 LLM 给出模糊的替换目标，导致意外修改多处代码。

ListDirTool — 列出目录内容，对每个条目做 `stat`，用 [folder] 和 [file] 前缀区分类型。

命令执行

ExecTool — 执行 shell 命令。三层防护：

第一层，危险命令正则黑名单：

```
const DANGEROUS_PATTERNS: RegExp[] = [
```

```
  /rm\s+(-[a-zA-Z]*f[a-zA-Z]*\s+)?(-[a-zA-Z]*r[a-zA-Z]*\s+)?\/($|\s)/,
  /rm\s+-[a-zA-Z]*rf?\s+~($|\/|\s)/,
  /mkfs\b/,
  /dd\s+if=/,
  /:\(\)\s*\{\s*:\|:\s*&\s*\}\s*;/,  // fork bomb
  />\s*\/dev\/[sh]d[a-z]/,            // 写入裸设备
  /chmod\s+-R\s+777\s+\//,
];
```

覆盖 `rm -rf /`、fork bomb、写裸设备等高危模式。需要说明的是，正则黑名单是最低限度的防线，不能替代沙箱隔离。

第二层，资源限制：默认30 秒超时，2MB `maxBuffer`。超时后进程被 kill，返回超时提示。

第三层，输出截断。超过 10,000 字符时取首尾各 5,000 字符，中间用截断标记连接：

```
function truncateOutput(text: string): string {
  if (text.length <= MAX_OUTPUT_LENGTH) return text;
  const half = Math.floor(MAX_OUTPUT_LENGTH / 2);
  return (
    text.slice(0, half) +
    `\n\n--- truncated (${text.length} chars total) ---\n\n` +
    text.slice(-half)
  );
}
```

保留首尾而非只取前 N 字符，是因为命令输出的末尾通常包含最有价值的信息（错误信息、统计摘要等）。

Web 能力

WebSearchTool — 封装 Brave Search API，返回结构化搜索结果。参数 `count` 可选，默认 5 条，上限 10 条。

WebFetchTool — 抓取 URL 内容。针对 HTML 页面内置了纯正则实现的 `htmlToText` 转换：

```
function htmlToText(html: string): string {
  return html
    .replace(/<script[\s\S]*?<\/script>/gi, "")
    .replace(/<style[\s\S]*?<\/style>/gi, "")
    .replace(/<(br|\/p|\/div|\/li|\/tr|\/h[1-6])[^>]*>/gi, "\n")
    .replace(/<[^>]+>/g, "")
    // HTML 实体解码...
    .replace(/[ \t]+/g, " ")
    .replace(/\n{3,}/g, "\n\n")
    .trim();
```

```
}
```

没有使用 DOM 解析库（如 cheerio、jsdom），纯正则处理。对于大多数常规网页足够用，但对复杂嵌套结构可能丢失语义。内容超过 20,000 字符时截断。

通信与调度

MessageTool — 出站方向的消息通道。通过构造时注入的 `sendCallback` 向外部发送消息：

```
export class MessageTool extends Tool {
  constructor(private sendCallback: SendCallback) {
    super();
  }
  async execute(args: Record<string, unknown>): Promise<string> {
    const content = args.content as string;
    const channel = (args.channel as string) ?? "repl";
    const chatId = (args.chat_id as string) ?? "default";
    await this.sendCallback({ channel, chatId, content });
    return `Message sent to ${channel}:${chatId}`;
  }
}
```

REPL 场景下`sendCallback` 就是 `console.log`；Bot 场景下替换为向 Telegram、Discord 等平台发送消息的函数。工具本身不关心消息最终去向。

CronTool + CronService — 定时任务管理，分为服务层和工具层。

CronService 基于 `setInterval` 实现，支持两种定时方式：`every_seconds`（直接转换为毫秒间隔）和 `cron_expr`（解析 cron 表达式为近似间隔）。

cron 表达式的解析是简化版本：

```
private parseCronInterval(expr: string): number {
  const parts = expr.trim().split(/\s+/);
  if (parts.length !== 5) return 60_000;
  const [minute, hour] = parts;
  // */N * * * * → 每 N 分钟
  if (minute?.startsWith("*/") && hour === "*") {
    const n = parseInt(minute.slice(2), 10);
    if (!isNaN(n) && n > 0) return n * 60_000;
  }
  // 0 */N * * * → 每 N 小时
  if (minute === "0" && hour?.startsWith("*/")) {
    const n = parseInt(hour.slice(2), 10);
    if (!isNaN(n) && n > 0) return n * 3600_000;
```

```
  }
  if (minute === "*" && hour === "*") return 60_000;      // 每分钟
  if (minute === "0" && hour === "*") return 3600_000;    // 每小时
  if (minute === "0" && hour === "0") return 86400_000;   // 每天
  return 60_000; // 复杂表达式降级为每分钟
}
```

只处理 */N、每小时、每天等常见模式。不支持"每周三 14:30"这类精确时间点的 cron 语义——setInterval 本身也做不到这件事。复杂表达式静默降级为每分钟执行一次，这是一个已知的精度妥协。

CronTool 对外暴露 add、list、remove 三个 action，作为 CronService 的 function calling 接口。



MessageBus：入站消息总线

MessageBus 处理入站方向的消息流——从子系统或外部流向主 Agent。

```
export class MessageBus {
  private listeners = new Map<string, Set<MessageHandler>>();
  private queue: InboundMessage[] = [];
  subscribe(channel: string, handler: MessageHandler): () => void {
    if (!this.listeners.has(channel)) {
      this.listeners.set(channel, new Set());
    }
    this.listeners.get(channel)!.add(handler);
    return () => { this.listeners.get(channel)?.delete(handler); };
  }
  async publish(message: InboundMessage): Promise<void> {
    const handlers = this.listeners.get(message.channel);
    if (handlers && handlers.size > 0) {
      for (const handler of handlers) {
        await handler(message);
      }
    } else {
      this.queue.push(message);
    }
  }
  drain(channel?: string): InboundMessage[] {
    if (!channel) {
      const msgs = [...this.queue];
      this.queue = [];
      return msgs;
```

```
    }
    const matched = this.queue.filter((m) => m.channel === channel);
    this.queue = this.queue.filter((m) => m.channel !== channel);
    return matched;
  }
}
```

数据结构 InboundMessage 包含四个字段：channel（消息通道）、senderId（发送者标识）、chatId（关联会话，格式为 originChannel:originChatId）、content（消息内容）。

两种消费模式：

1. **subscribe**— 注册实时回调。消息到达时立即调用 handler。适合常驻服务场景。
2. **drain**— 从队列中取出并清空消息。适合轮询式的同步消费场景。

路由规则：有订阅者走回调，无订阅者入队列。消息只走一条路径，不会同时触发回调和入队。

与 MessageTool 的关系需要明确：MessageTool 负责出站（Agent → 外部），MessageBus 负责入站（外部/子系统 → Agent）。两者没有直接的代码耦合，方向相反。



架构

单进程并发模型。每个子 Agent 是一个 Promise，共享同一个 Node.js 事件循环。没有多进程、没有 Worker。

每个子 Agent 拥有独立的 AgentLoop 实例，有自己的 ReAct 循环，没有历史上下文——每次从零开始，处理完一个任务就结束。

子 Agent 的工具集是主 Agent 的受限子集。在 index.ts 中通过 exclude 排除了 spawn、message、edit_file、cron：

```
const subagentTools = tools.exclude(["spawn", "message", "edit_file", "
```

排除 spawn 防止子 Agent 递归创建子 Agent；排除 message 防止子 Agent 直接向用户发消息（应该通过 MessageBus 回传给主 Agent 处理）；排除 edit_file 限制子 Agent 的写入能力；排除 cron 避免子 Agent 创建定时任务。

生命周期

```
spawn(params: { task: string; label?: string; ... }): string {
  const id = `subagent-${++this.counter}`;
  const label = params.label ?? `Task ${this.counter}`;
  const promise = this.runSubagent(id, params.task, label, ...);
  promise.finally(() => {
    this.runningTasks.delete(id);
  });
  this.runningTasks.set(id, { id, label, promise });
  return id;
}
```

流程：`spawn()` 分配自增 ID → 启动 `runSubagent()` 返回 Promise → 立即返回 ID。调用方不需要等待子 Agent 完成。

`runSubagent` 创建一个独立的 `AgentLoop`，`buildMessages` 只传入当前任务，不带历史：

```
buildMessages: (_history, userMessage) => [
  { role: "user" as const, content: userMessage },
],
```

子 Agent 最大迭代 15 次（主 Agent 是 10 次）。完成后通过 `bus.publish()` 将结果发送到 `system` channel：

```
await this.bus.publish({
  channel: "system",
  senderId: "subagent",
  chatId: `${originChannel}:${originChatId}`,
  content: `[Subagent "${label}" (${id}) completed]\n\n${result}`,
});
```

成功和失败都走这条路径，只是 content 不同。`promise.finally()` 负责从 `runningTasks` Map 中自动清理已完成的任务。

SpawnTool

SpawnTool 是主 Agent 触发子 Agent 的接口。LLM 通过 function calling 调用它，传入 `task`（任务描述）和可选的 `label`。返回值包含子 Agent ID 和当前运行中的子 Agent 数量，让 LLM 对并发状态有感知。



REPL（Read-Eval-Print Loop）是整个 Agent 的终端交互入口。用户在终端输入文本，Agent 处理后输出回复，循环往复。

启动流程

index.ts 的初始化按以下顺序执行：

1. 创建 Anthropic 客户端和 MessageBus 实例。
2. 注册所有工具到 ToolRegistry，区分主 Agent 工具集和子 Agent 受限工具集。
3. 初始化 CronService，触发回调通过 bus.publish() 写入 system channel。
4. 创建 SubagentManager，注册 SpawnTool（最后注册，因为依赖 SubagentManager 实例）。
5. 构建 ContextBuilder，加载 skills 和 memory。
6. 创建 AgentLoop 实例，使用 readline/promises 启动交互循环。

并发控制

核心问题：用户输入和子 Agent 回传结果都会触发 agent.run()，但 history 数组是共享的，不能并发修改。

解决方式——布尔互斥锁 + 暂存队列：

```
let processing = false;
const pendingSubagentResults: InboundMessage[] = [];
async function drainPendingResults(): Promise<void> {
  while (pendingSubagentResults.length > 0) {
    const msg = pendingSubagentResults.shift()!;
    const systemContent = [
      "[SYSTEM NOTIFICATION - Subagent Result]",
      msg.content,
      "Please summarize the above subagent result for the user.",
    ].join("\n\n");
    const reply = await agent.run(systemContent, history);
    history.push({ role: "user", content: systemContent });
    history.push({ role: "assistant", content: reply });
    console.log(`\nBot > ${reply}\n`);
  }
}
async function tryDrainPending(): Promise<void> {
  if (processing) return;
  processing = true;
  try {
    await drainPendingResults();
  } finally {
```

```
    processing = false;
  }
  rl.prompt();
}
```

processing 布尔标志充当互斥锁。同一时刻只有一个 agent.run() 在执行。子 Agent 结果到达时先 push 到 pendingSubagentResults 数组。tryDrainPending 只在 !processing 时进入，避免并发写入 history。

用户输入的处理流程：

```
rl.on("line", async (input) => {
  // ...
  processing = true;
  try {
    const reply = await agent.run(trimmed, history);
    history.push({ role: "user", content: trimmed });
    history.push({ role: "assistant", content: reply });
    await drainPendingResults();
  } finally {
    processing = false;
  }
  rl.prompt();
});
```

用户交互完成后，在 finally 释放锁之前，先调用 drainPendingResults() 处理期间积攒的子 Agent 结果。这保证了子 Agent 结果不会无限滞后。

消息订阅

```
bus.subscribe("system", (msg) => {
  pendingSubagentResults.push(msg);
  void tryDrainPending().catch(console.error);
});
```

system channel 是内部消息的统一入口。子 Agent 完成、CronService 触发，都通过 bus.publish() 发送到这个 channel。handler 只做两件事：入队、尝试消费。

每条子 Agent 结果被包装为 [SYSTEM NOTIFICATION] 格式注入 history，由主 Agent 总结后输出给用户。

MessageTool 的接线

REPL 场景下，sendCallback 直接输出到终端：

```
tools.register(
  new MessageTool((msg) => {
    console.log(`\n[Message → ${msg.channel}:${msg.chatId}] ${msg.conte
  }),
);
```

如果切换到 Bot 场景，只需要替换这个回调为向 Telegram、Discord 等平台发送消息的函数。工具系统和 Agent 逻辑不需要任何改动。




```
终端 stdin
   │
   ▼
REPL 主循环 (index.ts)
   │  ┌──────────────────────── history (共享, 互斥访问)
   │  │
   ▼  ▼
AgentLoop.run() ──→ Tool 调用
   │                  ├── 文件/命令/网络工具 → 直接返回结果
   │                  ├── SpawnTool → SubagentManager.spawn()
   │                  │                  └── 子 AgentLoop (独立 ReAct)
   │                  │                        └── bus.publish("system
   │                  ├── MessageTool → sendCallback → stdout
   │                  └── CronTool → CronService
   │                                    └── setInterval → bus.publish(
   │
   │  ◄── bus.subscribe("system") ◄── pendingSubagentResults 队列
   │
   ▼
stdout 输出
```

数据流有两条主线：

同步路径： 用户输入 → REPL → agent.run() → 工具调用 → 结果回传模型 → 最终回复 → stdout。这是标准的 ReAct 循环。

异步路径： SpawnTool / CronService → `bus.publish("system")` → handler 入队 → `tryDrainPending()` → `agent.run()` 处理系统通知 → stdout。异步结果通过 MessageBus 汇入主循环，由互斥锁保证不与同步路径冲突。



设计选择与局限

零框架依赖。不依赖 LangChain 等 Agent 框架，直接基于 Anthropic SDK 构建。好处是完全控制 API 交互细节，调试时不需要穿透框架抽象层。代价是部分基础能力需要自己实现。

schema 定义方式。运行时对象而非 Zod / JSON Schema 库。降低了复杂度和依赖数量，但缺乏运行时校验。如果 LLM 传入了格式错误的参数，错误只能在执行阶段暴露。

子 Agent 无持久记忆。每次 spawn 从零开始，适合一次性的并行任务（搜索、分析、计算）。不适合需要跨任务积累上下文的场景。

CronService 的 cron 表达式。近似实现，只支持常见的等间隔模式。复杂表达式会静默降级为每分钟执行，不会报错。如果需要精确的 cron 语义，应该引入 cron 解析库。

MessageBus 无持久化。纯内存队列，进程重启后队列消息丢失。对于 REPL 场景足够，Bot 场景如果需要消息可靠性，需要接入持久化存储。

ExecTool 安全边界。正则黑名单只是最低防线。LLM 可以通过变量展开、别名、管道组合等方式绕过正则检测。生产环境应该使用容器沙箱或受限用户执行。

REPL 并发模型。布尔锁在单用户场景下足够。Node.js 的单线程模型保证了 processing 标志不会出现竞态。但如果扩展到多用户（如 Bot 同时处理多个会话），需要每个会话独立的 history 和更完整的队列/锁机制。



总结

这个框架的核心设计可以概括为四个部分：

1. **Tool 抽象 + Registry 模式**— 统一的工具注册和调用接口，通过 `exclude()` 实现能力隔离。
2. **双通道消息机制**— 出站走 `sendCallback`（MessageTool），入站走 MessageBus，方向明确，互不耦合。
3. **Promise 并发的子 Agent**— 共享事件循环，独立 ReAct 循环，通过 MessageBus 回传结果。
4. **互斥锁驱动的 REPL 主循环**— 布尔标志 + 暂存队列，保证 history 的一致性。

这些模块组合成一个可扩展的 Agent 运行时。扩展新工具只需继承 `Tool` 并注册。切换接入层（REPL → Bot）只需替换 `sendCallback` 和输入源。子 Agent 的能力边界通过 `exclude()` 控制。

团队介绍

本文作者苏雄，来自淘天集团-会员技术团队。业务上，我们负责 88VIP、天猫积分、省钱卡、大会员、消费券等淘宝核心业务，同时支撑淘宝、千问、闪购等阿里业务的账号互联互通。技术上，我们深耕 AI 与业务融合，为消费者带来全新体验，为业务创造新增量。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法

大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 5155

留言

写留言

还没有留言，来写首评